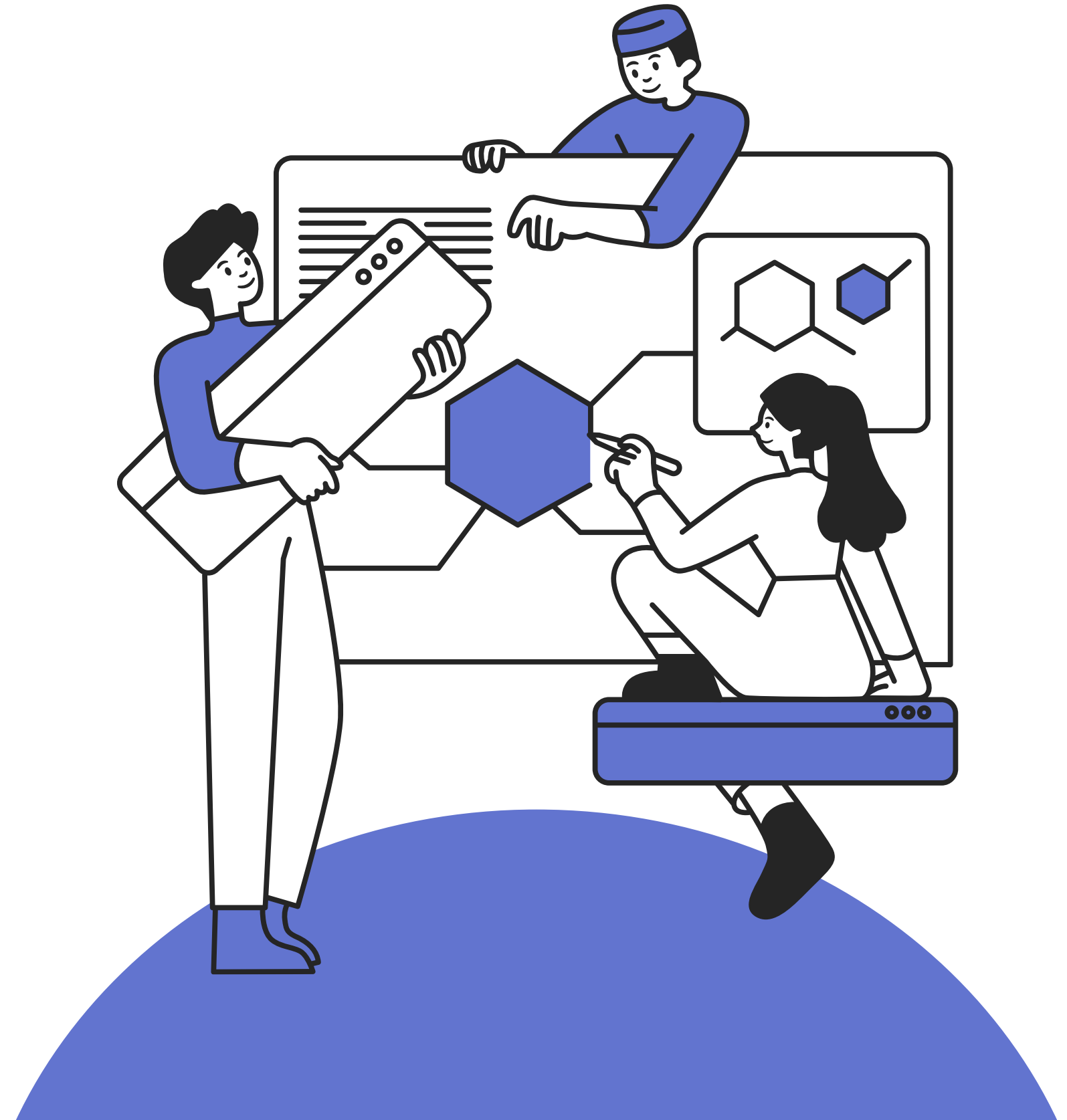


프로젝트 프레젠테이션

유진호 (2024254019)



CONTENTS

- 1 프로젝트 주제
- 2 주제 선정 동기
- 3 데이터셋 구성 및 전처리
- 4 모델 구조 및 처리 흐름도
- 5 적용 코드 및 실행 결과
- 6 앞으로의 계획
- 7 진행에 있어서 어려운 점
- 8 느낀점

01

프로젝트 주제

YOLO 기반 객체 탐지와 우선순위 판단 알고리즘을 통한 주의 유도 시각화 시스템

목표 : 이미지 또는 실시간 영상에서 객체를 탐지하고, 우선 순위를 판단하여 사용자가 주의해야 할 객체를 시각적으로 강조하는 시스템을 개발

탐지된 객체에 대해 우선순위 산정 › 우선순위에 따른 시각적 구분 틀 제공 › 원격/자율 주행 및 산업 안전 응용

02

주제 선정 동기



현업 연관성, 기술적 배경

- 자율주행 및 원격운전에 대한 연구와 기술 개발을 하고 있는 업무를 하고있으며,
자율주행 및 원격운전 기술이 발전함에 따라, 시각 정보 기반의 객체 인식 시스템 중요성 증가
- 단순 객체 탐지 이상의 객체 중요도 판단 기능이 필요함
- 사람의 시선처럼 “어디를 먼저 봐야 하는가?”를 알려줄 수 있는 주의 유도 시스템의 구현 가능성 탐색

03

사용 데이터 및 전처리 방법

데이터셋 구조



Train, val, test 등으로 구성

이미지 크기



640x480으로 정규화

객체 라벨링



Class, x_center, y_center,
width, height

03

사용 데이터 및 전처리 방법

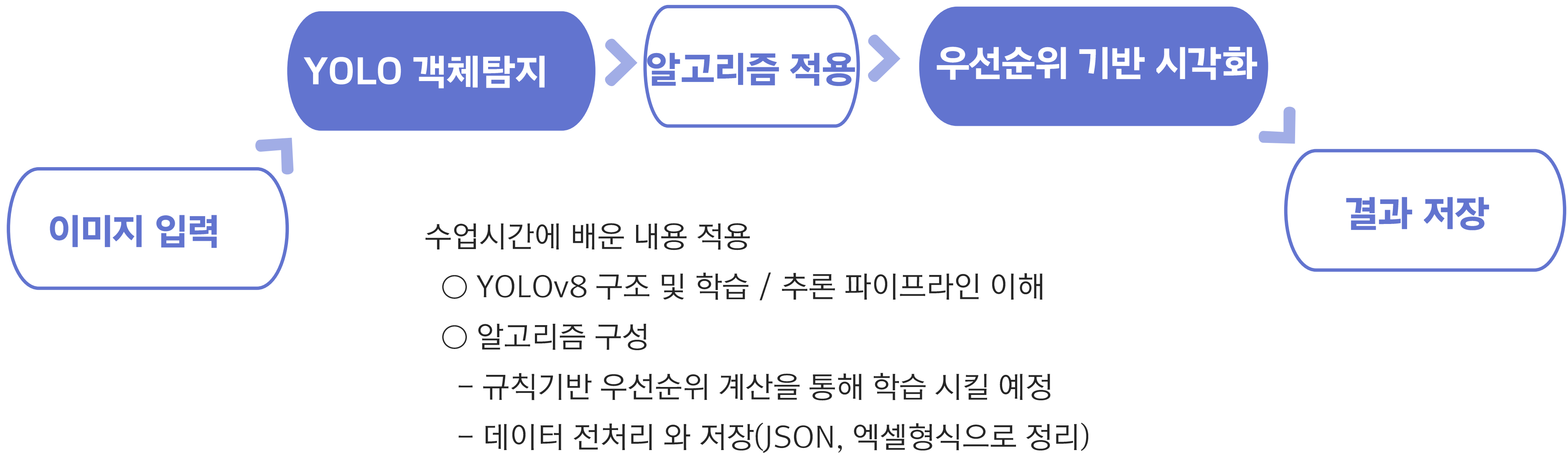
- 사용한 데이터 : COCO Dataset (YOLOv8 사전 학습 모델 사용)
 - > 총 80개의 클래스를 지원 (person, car, truck, dog 등)
 - > 초기에는 자체 이미지 데이터셋을 구축하려 했으나, 하나하나 시간이 오래걸려, 실제 거리 환경 이미지를 수집하여 YOLO 모델을 테스트
 - > 이미지 데이터셋 출처 : AI허브, Kaggle

*사용 이미지 약 2만장

- 향후 우선순위 판단 학습용 커스텀 데이터셋도 구축 예정

04

적용 알고리즘 및 흐름도



05

적용 코드 및 실행 결과

```
def compute_priority(cls_name, box, image_size):  
    weights = {  
        'person': 1.0,  
        'bicycle': 0.8,  
        'car': 0.6,  
        'bus': 0.5,  
        'truck': 0.5  
    }  
  
    cx, cy = (box[0] + box[2]) / 2, (box[1] + box[3]) / 2  
    iw, ih = image_size  
  
    center_dist = ((cx - iw/2)**2 + (cy - ih/2)**2)**0.5 / max(iw, ih)  
  
    box_size = (box[2] - box[0]) * (box[3] - box[1]) / (iw * ih)  
  
    base_weight = weights.get(cls_name, 0.3)  
  
    score = base_weight + 0.5 * box_size + 0.5 * (1 - center_dist)  
  
    return score
```

코드 설명

‘객체의 클래스, 위치, 크기에 따라 우선순위 점수를 계산’

클래스별 중요도 가중치 부여

이미지의 중심 좌표 계산

이미지 중심으로부터의 거리 정규화

박스 크기 비율 계산(넓이)

클래스 가중치

최종 점수 계산

05

적용 코드 및 실행 결과

```
1 from ultralytics import YOLO
2 import cv2
3 import os
4 import json
5 from priority_utils import compute_priority
6
7
8 model = YOLO('yolov8n.pt')
9
10 image_dir = './data'
11 save_dir = './results'
12 json_dir = './json'
13 os.makedirs(save_dir, exist_ok=True)
14 os.makedirs(json_dir, exist_ok=True)
15
16
17
18 for filename in os.listdir(image_dir):
19     path = os.path.join(image_dir, filename)
20     img = cv2.imread(path)
21
22     result = model(path)[0]
23
24     scored_boxes = []
25     json_data = []
26     image_size = img.shape[1], img.shape[0]
27
28     for box in result.boxes.data:
29         x1, y1, x2, y2, conf, cls = box.tolist()
30         cls_name = result.names[int(cls)]
31
32         score = compute_priority(cls_name, (x1, y1, x2, y2), image_size)
33         scored_boxes.append(((x1, y1, x2, y2), cls_name, score))
34
35         json_data.append({
36             "class": cls_name,
37             "bbox": [int(x1), int(y1), int(x2), int(y2)],
38             "image_size": list(image_size),
39             "priority": round(score, 4)
40         })
41
42 scored_boxes.sort(key=lambda x: x[2], reverse=True)
```

코드 설명

YOLOv8n 모델 적용

각각의 이미지 및 JSON 저장 디렉토리 설정

각각의 이미지마다 개별 처리 > YOLO 탐지 > 우선순위 계산 및 JSON 데이터 생성

점수 높은 순으로 정렬

05

적용 코드 및 실행 결과

```
for idx, ((x1, y1, x2, y2), cls_name, score) in enumerate(scored_boxes):
    color = (0, 255, 0)
    if idx == 0:
        color = (0, 0, 255)
    elif idx == 1:
        color = (0, 255, 255)

    cv2.rectangle(img, (int(x1), int(y1)), (int(x2), int(y2)), color, 2)
    cv2.putText(img, f'{cls_name}:{score:.2f}', (int(x1), int(y1) - 5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

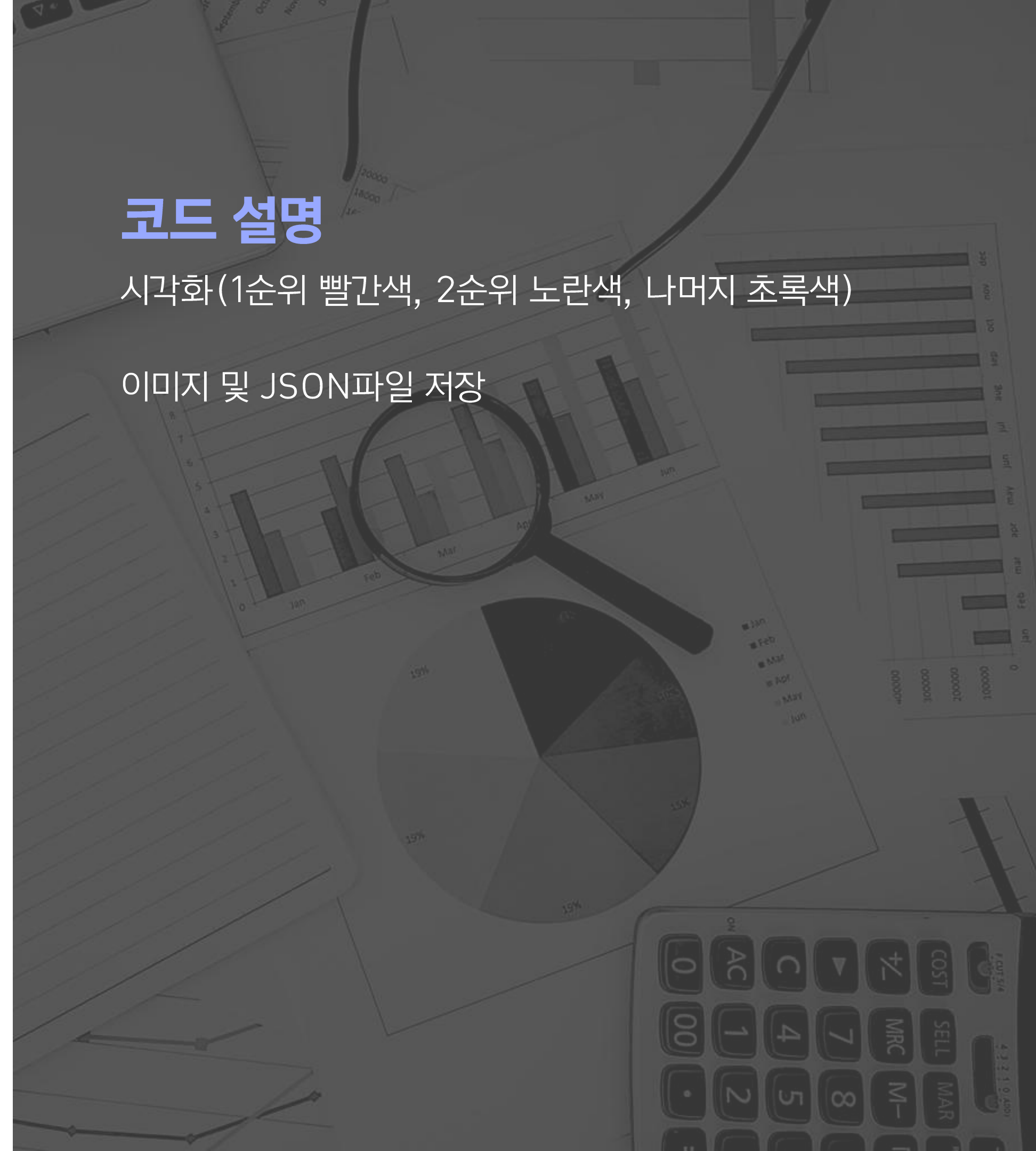
cv2.imwrite(f'{save_dir}/{filename}', img)

json_filename = os.path.splitext(filename)[0] + '.json'
with open(os.path.join(json_dir, json_filename), 'w') as f:
    json.dump(json_data, f, indent=2)
```

코드 설명

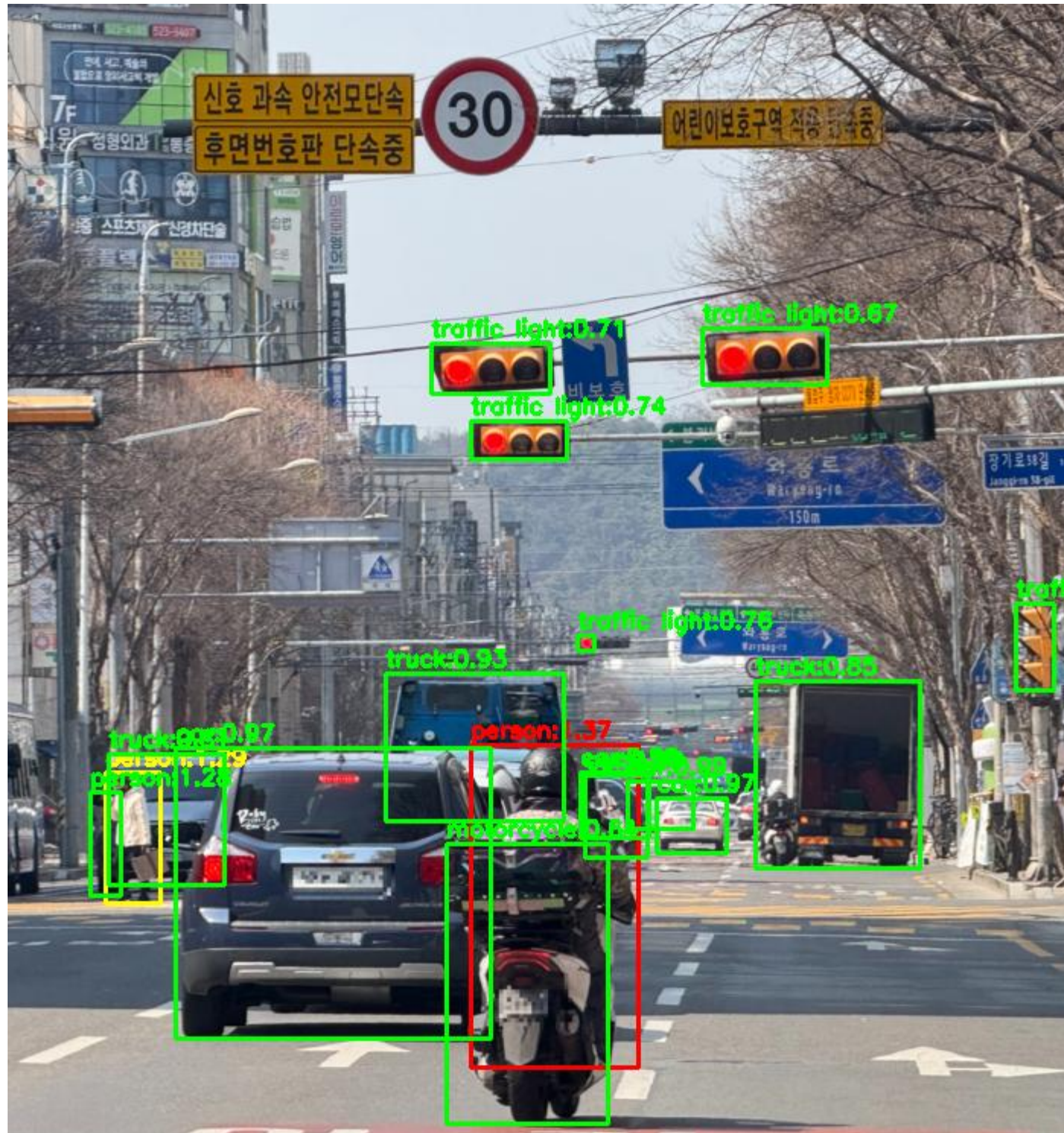
시각화(1순위 빨간색, 2순위 노란색, 나머지 초록색)

이미지 및 JSON파일 저장



05

적용 코드 및 실행 결과



실행 결과(잘된 예)

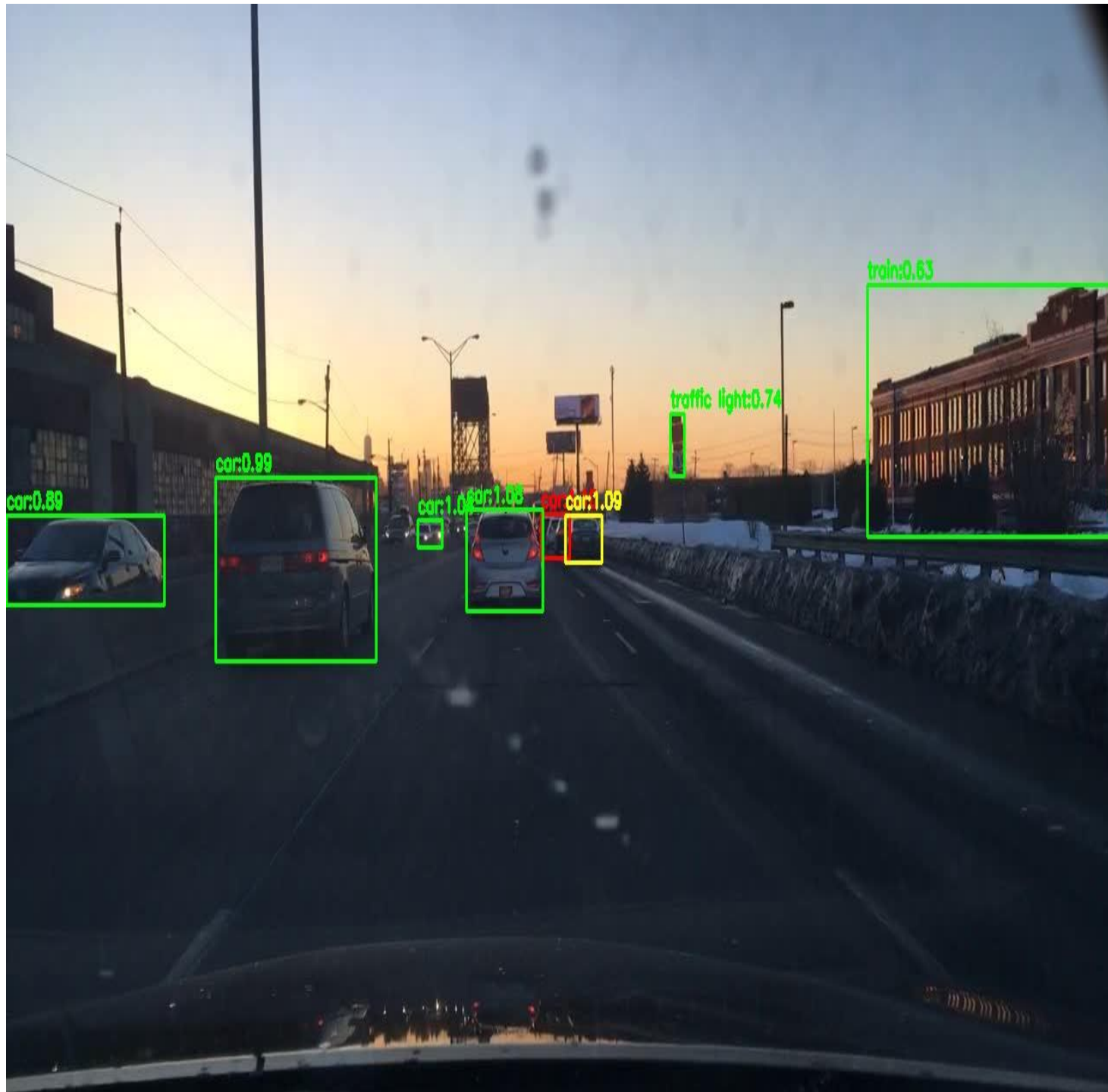
COCO데이터셋 사용으로 인해 신호등까지 탐지

Car, motorcycle, truck, person 각각의 다른 값의 가중치를 주었으며, 거리상 가장 가까운 사람인 오토바이 운전하는 사람이 1순위(빨간색)로 인식하고, 차보다 거리상 멀리 있지만 사람의 가중치가 높아 횡단중인 사람을 2순위(노란색)로 인식함.

- * COCO데이터 셋 중에 필요한 클래스만 탐지하도록
- 수정 필요

05

적용 코드 및 실행 결과



실행 결과 (잘못된 예)

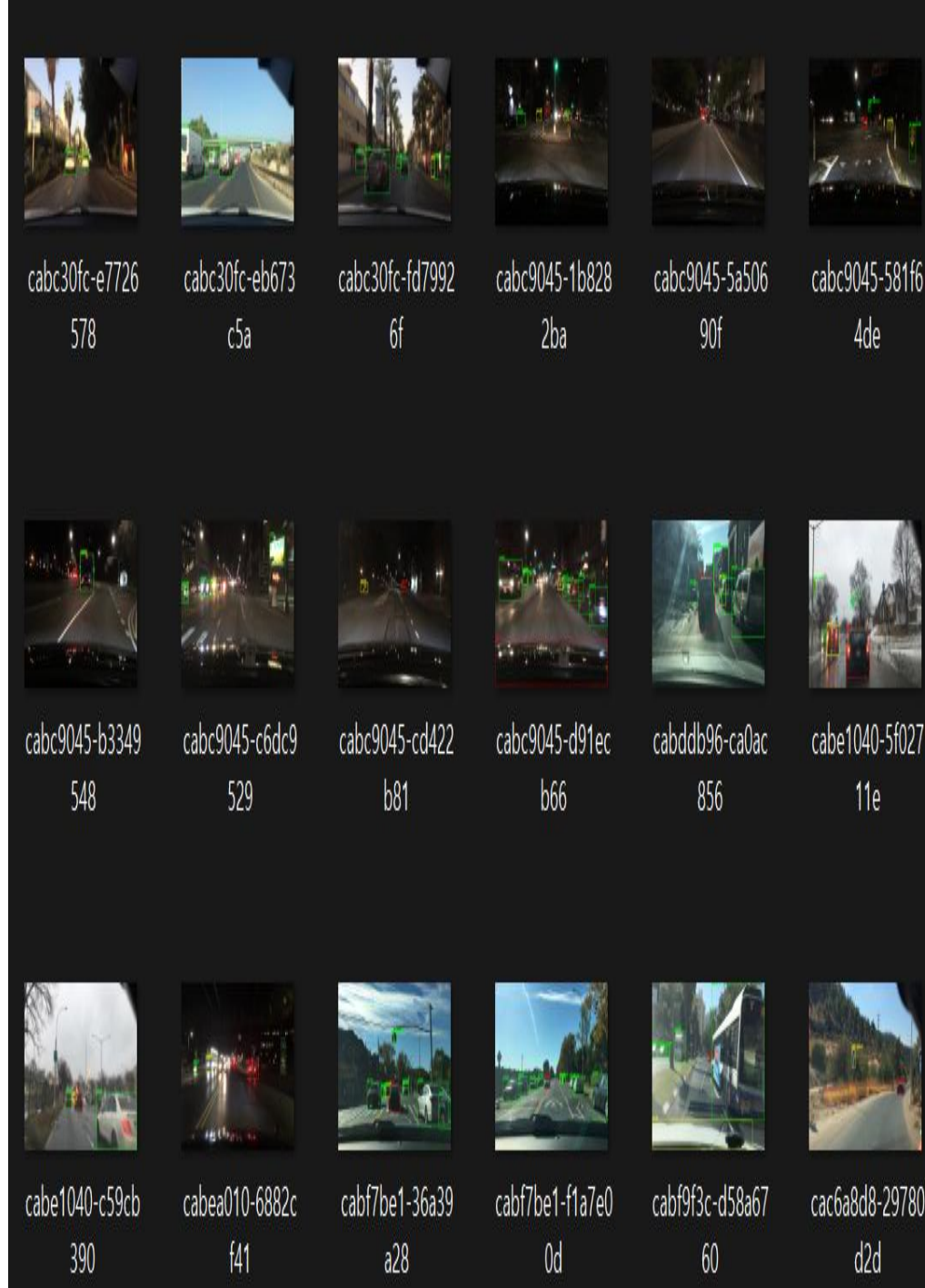
같은 차량이며, 거리가 더 가까운 차량이 있는데도 불구하고, 더 먼 차량이 우선순위가 되는 오류 발생

2만장의 이미지 데이터를 다 확인해보고 오류가 발생한 이미지들을 확인하여, JSON파일의 내용 수정 예정

05

적용 코드 및 실행 결과

```
{  
  "class": "car",  
  "bbox": [  
    470,  
    234,  
    527,  
    272  
  ],  
  "image_size": [  
    1280,  
    720  
  ],  
  "priority": 1.0322  
},  
{  
  "class": "car",  
  "bbox": [  
    250,  
    237,  
    343,  
    294  
  ],  
  "image_size": [  
    1280,  
    720  
  ],  
  "priority": 0.9639  
},
```



진행된 내용

YOLOv8 모델 탐지 및 결과 박스 시각화

객체마다 우선순위 계산 후 시각화(색상 차등 표시)

우선 순위 점수 기반 정렬 및 시각화 정확도 개선 중

06

앞으로의 계획

- 수집한 데이터 기반으로 YOLOv8 파인튜닝 진행
- 우선순위 판단 알고리즘을 딥러닝 기반 방식으로 확장(예 : MLP, GNN등)
- 이미지가 아닌 영상 입력 테스트 및 시스템 응답속도 측정
- 다양한 환경(야간, 복잡한 도로)에서의 테스트 수행 및 한계점 분석

07

진행에 있어서 어려운 점

기술적 어려움

- YOLO 결과에서 bbox/클래스값 직접 추출 방법 파악
- 우선순위 점수 정의와 정량화 기준 선정의 어려움
- 커스텀 모델 학습 위한 데이터 구성 시간 소요
- 시각화 위치 오차 및 다중 객체 처리 부분 보완 필요

개별적으로 느낀점

- 먼저 초기 발표한 프로젝트와 다른 주제로 보고드리게 되었는데, 국내의 표지판 데이터셋을 몇 천장, 몇 만장 단위로 구할 수 가 없었다. 수업시간에 배운 YOLO 분야를 적용하기엔 아직 부족한점이 많아 학습된 모델을 사용한 점이 개인적으로 아쉬움이 컸다. 우선 전공도 마찬가지로 1학년 1,2학기에 딥러닝에 대해서 자세히 배운 과목이 없어서 다소 낮은 수준의 프로젝트를 진행하고 있는 것 같아 이 프로젝트를 계기로 MLP, GNN등을 공부하여 결과값을 학습시키는 모델을 만들고 싶다.

THANK YOU
